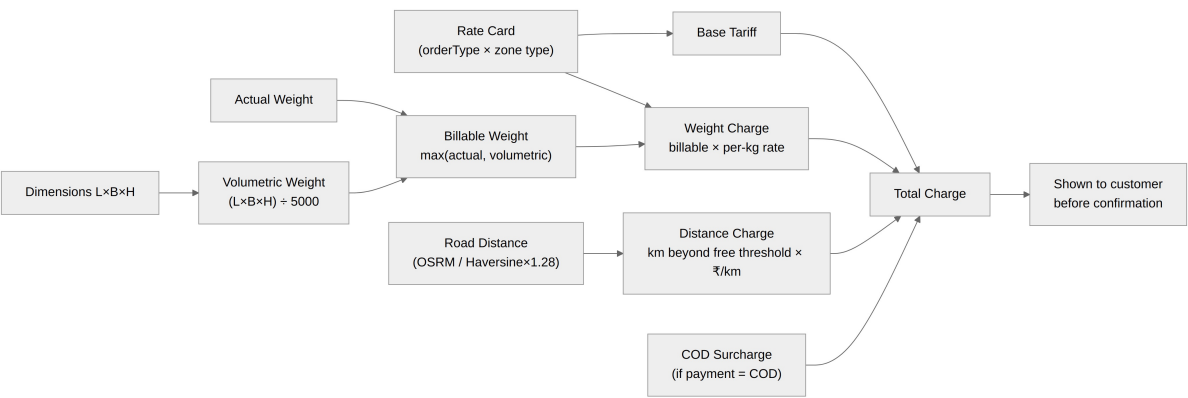


Delivero — System Design Write-Up

Rate calculation engine · zone detection · auto-assignment · failed delivery handling

1. Rate Calculation Engine

Delivero prices every order through a deterministic pipeline that runs entirely client-side in `orderService.preview()`, reading its inputs live from Firestore so nothing is hardcoded.



Billable weight. Volumetric weight is $(L \times B \times H \text{ in cm}) \div 5000$, the standard courier-industry formula. Billable weight is $\max(\text{actual weight}, \text{volumetric weight})$, so oversized-but-light parcels are charged fairly.

Rate card lookup. The engine reads the matching `rate_cards` document, keyed by `(orderType, rateType)` — B2B/B2C crossed with `INTRA_ZONE / INTER_ZONE` — giving a `baseCharge` and `perKgCharge`. If an admin hasn't configured a card yet, safe defaults (₹50/₹20 intra, ₹100/₹35 inter) keep the app functional on a fresh database.

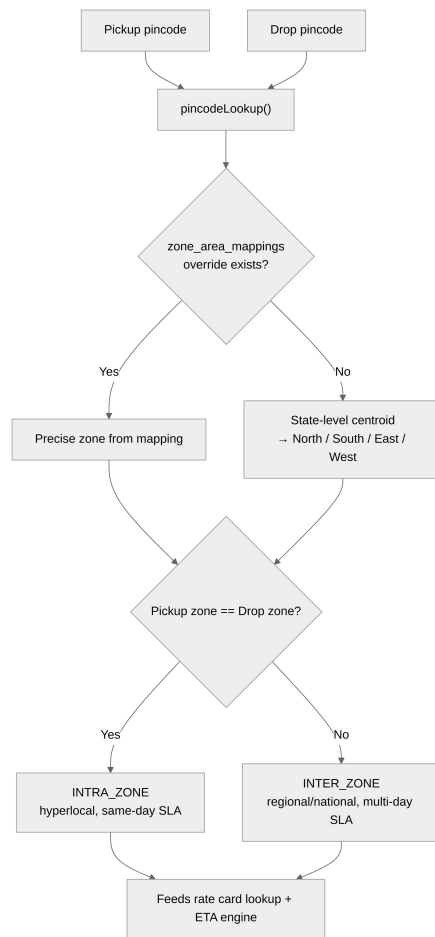
Distance charge. Real road distance is fetched from OSRM (falling back to Haversine $\times 1.28$ for Indian road winding if OSRM is unreachable). Intra-zone orders bill only the distance beyond a 5 km free radius at ₹5/km; inter-zone orders bill distance beyond 50 km at ₹3/km — reflecting that inter-zone jobs already carry a higher base tariff for linehaul.

Formula:

```
weightCharge    = billableWeight × perKgRate
distanceCharge  = max(0, distanceKm - freeKmThreshold) × ratePerExtraKm
baseCharge      = baseTariff + weightCharge + distanceCharge
codSurcharge    = admin-configured, per orderType (defaults ₹25 B2C / ₹40 B2B)
totalCharge     = baseCharge + codSurcharge
```

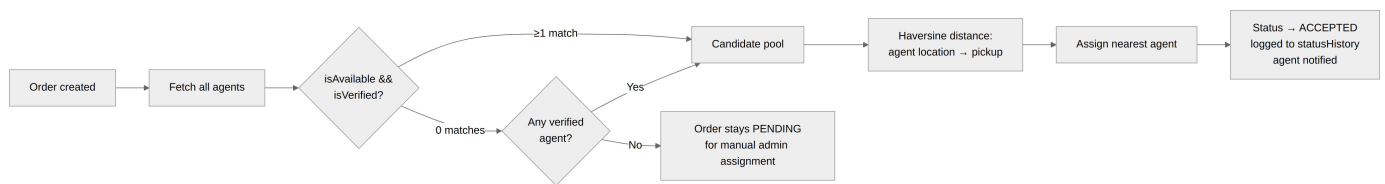
Every figure is rounded to two decimals and returned as a full breakdown, displayed to the customer before they confirm — so a rate-card change by an admin is reflected on the very next price preview with no redeploy.

2. Zone Detection Approach



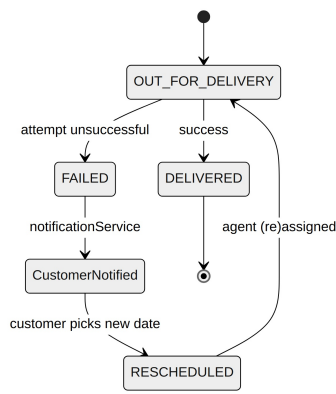
Each pincode is resolved through `pincodeLookup.ts`, which maps it to one of four zones with an approximate lat/long centroid. Admins can override this default resolution with precise entries in `zone_area_mappings`, so accuracy improves over time without a code change. Comparing the resolved pickup and drop zone names classifies the order `INTRA_ZONE` or `INTER_ZONE` — a single flag that drives both the rate card selected in §1 and the delivery-window promise (`computeLogisticsETA`): intra-zone orders get a same-day window, inter-zone orders get a 1-2+ business-day window scaled by distance.

3. Auto-Assignment Logic



On order creation the system fetches all delivery agents and filters to those flagged `isAvailable` and `isVerified`, falling back to “any verified agent” and then “any agent” if the strict filter comes up empty — so an order is never stranded unassigned while agents exist in the system. For each remaining candidate, the Haversine great-circle distance from their last recorded coordinates to the pickup point is computed, and the closest agent is assigned automatically; the order moves straight to `ACCEPTED`, the dispatch is written to `statusHistory`, and the agent is notified. Admins retain full manual control — they can assign an unassigned order directly or reassign any order at any time from the dashboard, which is the same mechanism reused during a reschedule.

4. Failed Delivery Handling



When an agent cannot complete a delivery, they mark the order **FAILED** with an optional note through `orderService.updateStatus` ; the transition is written immutably to `statusHistory` with a timestamp and the acting agent's ID. `notificationService` fires immediately, alerting the customer in-app and via push (FCM). The customer then submits a new delivery date through `orderService.reschedule` , which sets `status: RESCHEDULED` , stores the chosen date on the order, and logs the transition. The order re-enters the assignment flow so an agent — the same one or a reassignment from the admin dashboard — can attempt delivery again. Because every step is appended to `statusHistory` rather than overwriting a single field, the full failure-to-resolution journey stays auditable end to end.

(Multi-attempt caps, structured failure-reason codes, and automatic return-to-origin are natural extensions once the retry rate is measured in production — the current implementation supports one clean fail → notify → reschedule → retry cycle per attempt, repeatable as many times as needed.)